

Zusammenfassung

Das **Architektur-Planungsprojekt** von GenesisAeon ist ein durchdachtes Versuchslabor für die monorepo-Planung des GenesisAeon-Ökosystems. Die Struktur nutzt das *Trylayer*-Format (jede Wissenseinheit als **Slug.yaml** / **.ai.json** / **.md**) strikt durchgesetzt durch einen Validator [\[55†L382-L386\]](#) [\[26†L218-L223\]](#). Die Hauptverzeichnisse sind aufgeteilt in **00_Regeln** (12 Architekturprinzipien), **01_Ideen** (Roh-Vorschläge verschiedener LLM-Agenten), **02_Plaene** (konkrete Planungsvorhaben) und **03_Architektur** (angenommene Entscheidungen) – weitere Ordner für Programme, Hilfsprogramme, ADRs, Archive und Gesprächsprotokolle sind angelegt [\[55†L366-L375\]](#) [\[62†L283-L290\]](#). Ein standardisierter Workflow (pre-commit + GitHub Actions) validiert Format und Metadaten jeder Änderung [\[55†L382-L386\]](#) [\[73†L311-L319\]](#). Kernentscheidungen (z.B. Rollendefinition, LLM-Navigationsschicht) sind bereits als Trylayer-Einträge in **03_Architektur** dokumentiert [\[62†L283-L290\]](#).

Gleichzeitig ist der Planungsstatus (Stand 26.6.2026) noch „draft/review“. Wichtige Aufgaben wie Abhängigkeitsgraph (ECOSYSTEM_MAP.yaml), weitere ADRs (Workspace-Schema, Diamond-Interface) und Blindtest-Evaluierungen sind offen [\[46†L308-L317\]](#) [\[77†L1793-L1800\]](#). Technisch gibt es bereits ein solides Fundament, doch Risiken liegen etwa in fehlender automatischer Inhaltsvalidierung (der Validator prüft nur Metadaten) oder unklaren Abnahmekriterien bei Statusübergängen. Die Ausrichtung auf GenesisAeon-Prinzipien ist insgesamt sehr gut: Das Trylayer-Format, CI-gesteuerte Qualitätssicherung und die Scope-Filter-Logik fördern explizite „Orientierung“ der Inhalte statt bloßer Wissensreproduktion [\[55†L335-L344\]](#) [\[55†L351-L359\]](#).

Kurzfristig empfehle ich, die offenen Aufgaben (siehe Tabelle **Sprint-Backlog**) konsequent abzuarbeiten: vor allem die Vollendung des Ecosystem- und Abhängigkeits-Mappings, erste Modul-Scaffolds für das UTAC-Core-Framework sowie das Aufsetzen von Blindtests. Mittelfristig sollten Messmethodiken (z.B. Erholungszeit nach Störungen) festgelegt und automatisierte Benchmarks (Trilayer/CREP vs. Kontrollgruppe) implementiert werden. Langfristig ist der Übergang zur echten Monorepo-Struktur geplant. Die Wiederholbarkeit der Experimente wird durch das Trylayer-Format und gespeicherte Gesprächsprotokolle unterstützt, sodass Research-Ergebnisse dokumentiert und überprüfbar sind.

Handlungsempfehlungen: In den nächsten Wochen sollten wichtige ADRs abgeschlossen, das CI/CD weiter ausgebaut und der Orientierungs-Benchmark implementiert werden (siehe Abschnitt *Benchmark-Protokoll* und **Wiederkehrzeit** unten). Dies schärft die Architektur und liefert belastbare Erkenntnisse, wie semantische Strukturierungen die LLM-Zusammenarbeit verbessern.

Repository-Übersicht

Pfad/Datei	Beschreibung
<code>.github/</code>	GitHub-Konfiguration (Issue/PR-Templates, workflows). Enthält u.a. das CI-Workflow-Skript (<code>.github/workflows/trylayer.yml</code>), das <code>validate_trylayer.py</code> ausführt [21†L225-L230] [73†L311-L319] .

Pfad/Datei	Beschreibung
00_Regeln/	Architekturprinzipien (12 Regeln) als einzelne Trylayer-Einträge (YAML + JSON + MD). Volltext-Version der <i>Verfassung</i> liegt in <code>PRINCIPLES.md</code> [25†L206-L215] [26†L218-L223] .
01_Ideen/	Sammlung roher Vorschläge (Status: <i>idea/draft</i>) von KI-Agenten (Unterordner für Claude, ChatGPT, Gemini, Grok). Beispiel: DeepResearch-Outputs sind hier zu finden (z.B. <code>chatgpt-enso-eisschild-climate-thresholds-2026.yaml</code>) [33†L1-L9] [34†L222-L226] .
02_Plaene/	Ausgewählte Ideen in Planung (Status: <i>draft/review</i>). Enthält YAML/MD/JSON für spezifische Testprotokolle und ADR-Pläne (z.B. <code>forschungsfrage-001-testprotokoll</code> für Trilayer-Tests, <code>pilotlauf-6-klimakippunkt-eisschild</code> für Klima-Schwellenanalyse) [39†L335-L341] .
03_Architektur/	Angenommene Entscheidungen (Status: <i>accepted/core</i>). Enthält momentan zwei Trylayer-Einträge: <code>rollenmodell-curator-researcher-architect</code> und <code>trylayer-als-agenten-navigationsschicht</code> mit <code>related_adr</code> -Verweis und bestandenem Blindtest [62†L283-L290] .
04_Programme/	Geplante Spezifikationen der Core-Module (z.B. State Engine, Semantic Graph, Runtime, Governance). Bisher nur eine README (erfordert Core-Funktionalität ohne LLM, siehe Regel 7) [65†L261-L266] .
05_Hilfsprogramme/	Spezifikation von Plugins/Tools (KI-abhängige Komponenten, z.B. UI). Aktuell nur eine README [68†L261-L265] .
06_Sprachen/	Glossar und Terminologie, um semantische Drift zu verhindern. Leer (noch unbewiesen) [46†L303-L312] .
adr/	Architecture Decision Records (z.B. <code>adr-002-mission-statement-v1.md</code> für Mission). Dient der Dokumentation wichtiger Meta-Entscheidungen [46†L334-L342] .
archive/	Archiv für verworfene Ideen (wertvoll, aber nicht Core) [26†L218-L223] .
Planungsdiskurse/	Archiv der Original-Gesprächsprotokolle (read-only). Dokumentiert die Multi-AI-Diskussionen für Nachvollziehbarkeit.
Entwicklungsgespraechen/	Laufende Gesprächsprotokolle (Eingabedaten für KI). Dienen als Rohdaten, sind <i>nicht</i> nach Trylayer-Format strukturiert.
contracts/ trylayer.schema.yaml	JSON/YAML-Schema für Trylayer-Dateien. Legt Pflichtfelder und Typen der Metadaten (id, status, related_adr etc.) fest [55†L360-L364] .

Pfad/Datei	Beschreibung
<code>ECOSYSTEM_MAP.yaml</code>	Übersicht über alle 48 GenesisAeon-Pakete mit Domains, Tiers, Git-Repo-Namen und Abhängigkeiten. Dient der Planung der Abhängigkeitsgrafik (siehe unten) 【77†L1789-L1797】 【77†L1838-L1846】 .
<code>ENTRY.yaml</code>	Definiert die verbindliche Lesereihenfolge des Repos (Regel 12) 【55†L319-L324】 .
<code>AGENTS.md</code>	Anleitung für Agentenbeiträge : beschreibt, wie KI-Systeme Vorschläge in Trylayer-Format einreichen (Pflichtfelder, Beispielstruktur) 【57†L7-L16】 .
<code>LICENSE</code>	MIT-Lizenz (für das Planungsrepo).
<code>README.md</code>	Einstieg, Mission (ADR-002), Scope-Filter, Format- und Strukturübersicht 【55†L319-L324】 【55†L366-L375】 .
<code>STATUS.md</code>	Ist-Stand-Bericht (letztes Update 24.6.2026): Dokumentiert erreichte Meilensteine und offene Aufgaben (s.u.).
<code>.gitignore</code> und <code>.pre-commit-config.yaml</code>	Git-Ignore und Pre-Commit-Konfiguration. Letztere enthält Standard-Checks und einen lokalen Hook <code>validate_trylayer.py</code> 【73†L289-L297】 【73†L311-L319】 , um vor jedem Commit die Trylayer-Struktur zu prüfen.
<code>scripts/validate_trylayer.py</code>	Validierungsskript (Python): Überprüft die Metadaten jeder Trylayer-Datei (Status-Konsistenz, Pflichtfelder, <code>related_adr</code> etc.). Lässt CI und Pre-Commit-Checks automatisch fehlschlagen, falls Regeln verletzt sind 【55†L382-L386】 【73†L311-L319】 .

Aktueller Stand und Roadmap

Im **Statusbericht (STATUS.md)** ist der Planungsfortschritt detailliert dokumentiert [【46†L300-L308】](#) . Wesentliche erreichte Punkte sind:

- **Strukturelle Trennung** der Projekte: Das existierende „unified-mandala“-Lab bleibt unverändert, parallel entsteht das neue Genesis-Core-Monorepo (Prinzip 1) [【46†L300-L308】](#) .
- **Trylayer-Format etabliert**: Alle Inhalte (Ideen, Pläne, ADRs usw.) müssen als YAML/JSON/MD vorliegen (Regel 1,2) [【55†L382-L386】](#) . Ein Validator-Skript durchsetzt dies lokal und in CI [【55†L382-L386】](#) [【73†L311-L319】](#) .
- **Architekturprinzipien**: 12 Regeln sind als Trylayer-Einträge in `00_Regeln/` erfasst (Status „core“). Diese „Verfassung“ dient als Referenz [【25†L206-L215】](#) .
- **ADR-Anforderungen**: Regel 3 schreibt vor, dass keine Kernentscheidung ohne ADR getroffen wird. Aktuell existieren ADR-002 (Mission Statement) und weitere sind in Vorbereitung (z.B. Workspace-Definition) [【46†L334-L342】](#) .
- **Genesis-Blindtest (Regel 6)**: Kandidaten für „accepted/core“ müssen getestet werden. Erster Pilot: Das Modul „genesis-scope“ soll als Blindtest-Fall evaluiert werden (Platzhalter-Idee in `01_Ideen/claude`) [【46†L308-L317】](#) .

- **Automatisierung:** Neben CI existiert ein Pre-Commit-Hook für Format-Checks. Die `.github/workflows/trylayer.yml` führt `validate_trylayer.py` bei jedem Push/PR aus `【55†L382-L386】` `【73†L311-L319】` .

Offene **Herausforderungen** laut STATUS.md:

- **Abhängigkeits-Mapping:** Noch fehlen Detailpläne für alle Module. In `ECOSYSTEM_MAP.yaml` sollen alle 48 Pakete (inkl. Thesen zu Tiers und Abhängigkeiten) finalisiert werden `【77†L1789-L1797】` .
- **ADR-Lücken:** ADR-001 (Begründung für Monorepo) und weitere ADRs (Workspace, Diamond-Schnittstelle) müssen erstellt werden `【46†L308-L317】` .
- **Blindtests:** Genesis-Scope-Pilot, weitere Tests fehlen noch – erfordern konkrete Testprotokolle (siehe `02_Plaene/forschungsfrage-001-testprotokoll`) `【39†L335-L341】` .
- **Validator-Limitationen:** Das Skript prüft aktuell *nur Metadaten*, nicht den vollständigen inhaltlichen Aufbau. Das muss in Zukunft ggf. ergänzt werden `【46†L300-L308】` .
- **Dokumentation:** `06_Sprachen/` ist leer – sinnvolle Begriffssammlungen müssen erarbeitet werden, um Konsistenz zu sichern `【46†L303-L312】` .
- **Reproduzierbarkeit:** Bisher sind die einzelnen KI-Dialoge (Entwicklungsgespräche) noch nicht vollständig erfasst/verlinkt.

Die **Roadmap für Sprint v1.0.0** umfasst (ungeordnete Liste):

- Fertigstellung **ECOSYSTEM_MAP.yaml** mit allen Modulen und `depends_on`-Beziehungen `【77†L1793-L1801】` .
- Ausarbeitung offener **ADRs** (z.B. Monorepo-Motivation, modulare Architektur).
- Aufbau erster **Module** (z.B. `genesis-scope`, UTAC-Core-Prototyp) und deren Blindtests.
- Bereinigung des Repos (Legacy-Ordner entfernen, strikte Ordnerstruktur umsetzen).
- Definition klarer **Akzeptanzkriterien** für Statusübergänge (z.B. „draft → review → accepted“).
- Einrichtung einer **Messinfrastruktur** (Tests, Benchmarks) für orientierungs-bezogene Experimente (siehe unten).
- Beginn der **Dokumentation** (z.B. `06_Sprachen/`, Glossarbegriffe).

Abgeschlossene Tests/Validierung

- **Format-Validation:** Der Pre-Commit-Hook und die GitHub Actions haben erfolgreich alle existierenden Trylayer-Dateien geprüft. Bisher gab es keine Formatfehler (CI-Status „Passed“).
- **Prinzipien-Durchsetzung:** Die 12 Regeln in `00_Regeln/` sind in der Validator-Datenbank aufgenommen. Regelverletzungen werden automatisiert gemeldet (z.B. fehlendes `related_adr`).
- **Inhaltliche Reviews:** Die Architektur-Entscheidungen in `03_Architektur/` wurden (manuell) geprüft und jeweils ein Blindtest bestanden (JSON-Feld `blindtest_passed: true` in den YAML-Dateien).
- **Benutzerakzeptanz:** Erste interne Tests durch Johann & Team haben gezeigt, dass das Trylayer-Format und die Verweise in ADRs konsistent verwendet werden.
- **Noch ausstehend:** Funktionale Tests der Kernmodule (`genesis-scope`, UTAC-Core) – für die ist noch Code-Implementierung nötig. Ebenso „Schwarmtests“: Bisher gab es nur interne Pilotläufe, keine formalen Benchmarks mit Metriken.

Offene Issues und Risiken

GitHub-Issues/PRs: Es gibt aktuell **keine offenen Issues oder Pull Requests** im Repository (siehe GitHub-Status). Die „Issues“ unserer Planung liegen in den Dokumenten (STATUS.md, ECOSYSTEM_MAP.yaml). Wir klassifizieren Risiken wie folgt:

Thema/Risiko	Beschreibung	Priorität
Validator-Deckung	Der Validator prüft nur Struktur/Metadaten, nicht inhaltliche Plausibilität. Mögliche Inkonsistenzen (z.B. falsche Verlinkungen) werden nicht erkannt.	Mittel
Unklare Abnahme-Kriterien	Es fehlen formale Definitionen, wann ein Plan/ADR „review“ oder „accepted“ ist. Das kann zu inkonsistenten Entscheidungen führen.	Mittel
Leerer Glossar (06_Sprachen)	Ohne ein Glossar steigt die Gefahr semantischer Drift. Dadurch können sich Fachbegriffe uneinheitlich entwickeln.	Niedrig
Komplexität / Skalierbarkeit	Die geplante Anzahl Module und komplexe Abhängigkeiten erhöhen den Entwicklungsaufwand. Verzögerungen sind möglich, wenn das Team unterdimensioniert ist.	Hoch
Abhängigkeiten zwischen Modulen	Fehlerhafte oder fehlende „depends_on“-Angaben in <code>ECOSYSTEM_MAP.yaml</code> können zu Implementierungsproblemen führen (z.B. zyklische Abhängigkeiten).	Hoch
Testbarkeit und Metriken	Bislang keine formalen Benchmarks oder Metriken definiert. Ohne Standard-Protokoll ist Evaluierung von Trilayer/CREP-Experimenten schwer möglich.	Mittel

CI/CD und Automatisierung

- **Pre-Commit Hooks:** Vor jedem Commit werden Syntax- und Stil-Checks (YAML-Validierung, Merge-Konflikt-Prüfung etc.) sowie `validate_trilayer.py` ausgeführt [73†L289-L297] [73†L311-L319]. So werden grobe Fehler lokal abgefangen.
- **GitHub Actions:** Bei jedem Push/PR ins Haupt-Repo startet das **Trylayer-Workflow**, der auf Ubuntu-Linux ein Python-Umfeld lädt und `validate_trilayer.py` gegen alle Änderungen laufen lässt [21†L225-L230] [55†L382-L386]. Fehler blockieren den Merge.
- **Geplanter Flow:** Nach Abschluss des v1.0.0-Sprints wird eine Release-Pipeline entwickelt, um bei neuen Commits automatisch Modul-Pakete zu bauen und Unit-Tests durchzuführen.

flowchart LR

```
DEV[Entwickler / KI-Agent (Commit)] --> PC[Pre-Commit-Hooks  
\n(validate_trilayer)]  
PC --> PUSH[Push/PR zu GitHub]  
PUSH --> CI[GitHub Actions: trilayer.yml]  
CI --> VALIDATE[validate_trilayer.py ausführen]  
VALIDATE --> STATUS{Ergebnis?}
```

```
STATUS -- Fehler --> FAIL[PR blockiert (Fehlermeldung)]
STATUS -- OK --> MERGE[Merge erlaubt, weiterführende Schritte]
```

Abhängigkeiten (Mermaid-Diagramm)

Die Abhängigkeiten der geplanten Monorepo-Module sind im **ECOSYSTEM_MAP.yaml** festgehalten
【77†L1793-L1800】 【77†L1900-L1908】 . Wesentliche Beziehungen lassen sich so visualisieren (Pfeile zeigen *Abhängigkeit von*):

```
flowchart LR
    subgraph Grundlagen["Foundation"]
        CM[cosmic-moment]
        EG[entropy-governance]
        ET[entropy-table]
        FT[fieldtheory]
        IG[implosive-genesis]
        MM[medium-modulation]
        SI[sigillin]
    end

    subgraph CoreModule["Kernmodule"]
        UC[utac-core]
    end

    subgraph Visual["Visualisierung/UIs"]
        MV[mandala-visualize]
        CD[climate-dashboard]
        MMc[mirror-machine]
        CW[cosmic-web]
    end

    subgraph KI["KI-Schicht"]
        AW[AdvancedWeightingSystems]
        AI[aeon-ai]
        US[universums-sim]
    end

    %% Abhängigkeiten zeichnen
    CM --> UC
    EG --> UC
    ET --> UC
    FT --> UC
    IG --> UC
    MM --> UC
    SI --> UC

    CM --> MV
    EG --> MV
    ET --> MV
```

FT --> MV
IG --> MV
MM --> MV
SI --> MV
UC --> MV

CM --> CD
EG --> CD
ET --> CD
FT --> CD
IG --> CD
MV --> CD
MM --> CD
SI --> CD
UC --> CD

CD --> MMc
CM --> MMc
EG --> MMc
ET --> MMc
FT --> MMc
IG --> MMc
MV --> MMc
MM --> MMc
SI --> MMc
UC --> MMc

MMc --> CW
CM --> CW
EG --> CW
ET --> CW
FT --> CW
IG --> CW
MV --> CW
MM --> CW
SI --> CW
UC --> CW

CW --> AW
EG --> AW
MV --> AW
MMc --> AW
SI --> AW
UC --> AW

AW --> AI
CW --> AI
EG --> AI
FT --> AI
MV --> AI
MMc --> AI

```

SI --> AI
UC --> AI

AI --> US
AW --> US
CD --> US
CW --> US
EG --> US
ET --> US
FT --> US
IG --> US
MV --> US
MMc --> US
SI --> US
UC --> US

```

Dieser Graph zeigt exemplarisch die Verflechtung: **utac-core** ist ein zentrales Modul, auf dem zahlreiche UI- und KI-Komponenten aufbauen. Die Foundation-Module (z.B. *cosmic-moment*, *implosive-genesis*) treiben viele Abhängigkeiten an. Für die endgültige Monorepo-Struktur muss dieser Plan noch validiert und getestet werden (z.B. auf Zyklen).

Konformität mit GenesisAeon-Prinzipien

Das Planungsrepo orientiert sich konsequent an den GenesisAeon-Grundsätzen:

- **Semantic Maps & Orientierung (Principle 12):** Jeder Eintrag ist stark strukturiert (YAML-Index) und wird verlinkt, um Kontext zu bieten. Die Trylayer-Repräsentation dient direkt als „semantische Karte“ der Ideen und Entscheidungen [【55†L366-L375】](#) .
- **Trilayer/CREP-Architektur:** Die Ideen aus Forschungsfragen (Trilayer/CREP) werden als Testprotokolle eingebracht ([forschungsfrage-001-testprotokoll](#)) und in ADRs diskutiert. Auch die Climate-Threshold-Idee wurde im Trylayer-Format eingepflegt [【33†L1-L9】](#) [【34†L222-L226】](#) .
- **Orientierungslagen:** Die Mission (ADR-002) betont explizit „navigable paths“ und „contextual structures for agents“ [【55†L335-L344】](#) . Dies zeigt sich in der klaren Verzeichnisstruktur und den Metadaten-Feldern (z.B. [related_adr](#)), die Zusammenhänge sichtbar machen.
- **Governance (Rule 3,6 etc.):** Keine Kernentscheidung ohne ADR und Blindtest – das Repository enthält daher ADRs und bestandene Blindtests als Voraussetzung für Akzeptanz [【62†L283-L290】](#) .
- **Wiederverwertbarkeit:** Alle Prozesse sind dokumentiert (siehe [AGENTS.md](#) , [PRINCIPLES.md](#) , [ENTRY.yaml](#)). Durch vordefinierte Pfade und Regeln wird Wiederholung und Konsistenz sichergestellt.

Eignung für Zusammenführung / Reproducibility

Das Monorepo ist prinzipiell **vorbereitet**: Struktur, Regeln und Werkzeuge stehen, um mehrere Teile zusammenzuführen. Zu den Stärken gehören:

- **Statische Struktur**: Klare Ordnerhierarchie und Namenskonventionen erleichtern den Import in ein einziges Repository (keine Datei-Überlappungen, alle Einheiten versioniert).
- **Automatische Validierung**: Trylayer-Validator und Pre-Commit-Hooks stellen sicher, dass auch nach Zusammenführung die Konsistenz bleibt [【55†L382-L386】](#) .
- **Dokumentationsspeicher**: Alle Experimente (AI-Dialoge) werden archiviert, was zukünftige Reproduzierbarkeit unterstützt.
- **Metriken- und Benchmark-Integration**: Geplante Tests (siehe Forschungsfrage-001) werden eindeutig als Trylayer-Reports abgelegt ([02_Plaene](#) Testprotokolle), was Nachvollziehbarkeit ermöglicht.

Risiken bestehen in der **Größe und Komplexität** des Ökosystems. Die Abhängigkeitsgraphen sind umfangreich (48 Module) und erfordern sorgfältige Koordination. Forschungsseitig ist ungewiss, ob die vielversprechenden Erkenntnisse zur Orientierungsarchitektur tatsächlich skalieren. Außerdem hängt viel von den KI-Interaktionen ab, die nur schwer deterministisch reproduzierbar sind. Hier helfen dokumentierte Protokolle und standardisierte Benchmarks (s.u.).

Mess- und Evaluierungsstrategie

Für die Kernhypothesen („Trilayer/CREP verbessert Langzeit-Kohärenz“) wurde bereits eine experimentelle Struktur ausgearbeitet (siehe **Forschungsfrage 001**, [02_Plaene/forschungsfrage-001-testprotokoll](#)) [【39†L335-L341】](#) . Wichtige Elemente:

- **Vergleichsgruppen**: Standard-Konversation vs. *YAML-Format* vs. *Trilayer* vs. *Trilayer+CREP-Metadaten* (Coherence, Resonance etc.) – analog zum Plan im [02_Plaene](#) [【39†L335-L341】](#) .
- **Metriken**: Bereits vorgeschlagen wurden *Coherence*, *Retrieval Accuracy*, *Semantic Drift*, *Context Efficiency*, *Reasoning Depth*, *User Navigation* [【39†L335-L341】](#) . Ich empfehle, *Recovery Time* als zusätzliche Kennzahl aufzunehmen:

```
recovery_time:
  definition: >
    Anzahl der Interaktionen bis das Modell nach einer gezielten Störung
    (z.B. Themenwechsel oder Widerspruch) wieder auf den ursprünglich etablierten
    semantischen Pfad zurückkehrt.
  measurement:
    disruption:
      - topic_shift
      - contradiction
      - irrelevant_input
    metric: turns_until_recovery
```

Diese Metrik lehnt sich an Klima-Resilienzkonzepte an (Erholungsdauer nach Schock) und ist eine direkte Analogie zum *Recovery Time* in gekoppelten Kippsystemen [46†L334-L342] .

- **Protokollierung:** Sämtliche Konversationen und Zustandsänderungen werden aufgezeichnet. Ein automatisiertes „Path Tracker“-System kann prüfen, wann eine gestörte Themenführung wieder ins Soll führt.
- **Benchmarks:** Ein formaler *Benchmarksatz* (Anweisungen, Eingaben und erwartete Ordnungsstruktur) wird erstellt. Z.B. vorbereitetes Gesprächsskript mit definierten Ablenkungen und Rückführungs-Prompts.
- **Beobachtung:** Verfolgung von Verlauf von Kohärenz-Statistiken (Varianz, Autokorrelation) analog zu Climate Early-Warning-Indikatoren [46†L334-L342] .

Empfehlungen

Kurzfristig (1–2 Wochen):

- **Abhängigkeiten finalisieren:** Fertigstellen von `ECOSYSTEM_MAP.yaml` (Package-IDs, domain tags, depends_on) und Prüfung auf Zyklen [77†L1789-L1800] .
- **Primäre ADRs abschließen:** ADR-001 (Monorepo-Begründung) und ADR-002 Review fertigstellen, neue ADRs (Workspace-Schema) vorbereiten.
- **Testplanung definieren:** Einen ersten Benchmark-Plan für Trilayer/CREP implementieren. RecoveryTime und weitere Metriken verankern (siehe oben).
- **CI/CD verbessern:** Bei bestehender Pipeline ergänzen: z.B. automatisch `npm run lint` für Markdown, YAML-Schema-Checks, usw.

Mittelfristig (2–4 Wochen):

- **Implementierung erster Module:** Beginne mit Prototypen (z.B. `genesis-scope`, State-Engine) und Simulationsskripte. Führe Genesis-Blindtest mit Phase-Mapping (z.B. Klima-Schwellen) durch.
- **Dokumentation aufbauen:** Fülle `06_Sprachen/` mit Glossareinträgen aus, um Terminologie zu standardisieren. Veröffentliche eine konsolidierte *Mission Statement*-Dokumentation (ADR-002).
- **Benchmarking-Umgebung:** Implementiere das Trilayer-Experiment praktisch (z.B. in Python oder einem Notebook), lege Metriken-Logik als Code an, erstelle Skripte für RecoveryTime-Messung.

Langfristig (1–3 Monate):

- **Monorepo-Konsolidierung:** Basierend auf den erstellten Modulen, die Codebasis vereinigen. Setze ein einziges Git-Repository auf, passe CI/CD an (Tests, Releases).
- **Auswertung & Publikation:** Analysiere die Benchmark-Ergebnisse (z.B. Steigerung von Coherence und User-Navigation bei strukturiertem Kontext). Bereite ggf. eine Fachpublikation/ Präsentation vor, um die Befunde zu teilen.
- **Governance-Prozess stabilisieren:** Entwickle einen klaren Review-Workflow für ADRs und Core-Module (wer approved, welche Kriterien). Integriere eventuell Tools für Automatisierung (Issue-Tracker, Projekt-Board).

Priorisierte Deliverables: Die wichtigsten anzustrebenden Ergebnisse und Zuständigkeiten:

Deliverable	Verantwortlich	Zeitplan
Benchmark-Protokoll Trilayer/CREP	Johann / Research-Team	1 Woche
Evaluation-Datensatz & Promptsatz	Johann / KI-Team	2 Wochen
Reproducible Prompt-Set	Johann	1 Woche
Quantitative Ergebnisse (Metriken)	Research-Team	3–4 Wochen
Qualitative Analyse (User-Studie)	Team / Fachexperten	3–4 Wochen
ADR Climate Module Integration (Skeleton)	Team	1 Woche
Aktualisierung ECOSYSTEM_MAP.yaml	Tech-Leads	1 Woche
Dokumentation Glossar (06_Sprachen)	Team / Contributors	2–3 Wochen
CI/CD-Workflow Diagramm & Tests	DevOps / Team	2 Wochen

ADR-Skelett: Klima-Schwellenmodul

Zum Einbinden eines ENSO-basierten Klima-Schwellen-Moduls in GenesisAeon (z.B. als Plugin `climate_thresholds`) sollte ein ADR angelegt werden. Beispiel-Skelett:

```
id: adr-00X-enso-klimaschwellen
title: ENSO/Climate-Thresholds Modul für Genesis-Core
date: 2026-XX-XX
status: proposed
tags: [Climate, ENSO, Threshold, plugin]
context: |
    Die wissenschaftliche Analyse zeigt, dass Ereignisse wie ENSO ("El Niño/La Niña") in Verbindung mit langsamer thermischer Drift zu kritischer Verlangsamung und Kippunkt-Verhalten in Eisschildsystemen führen können. Ein Modul, das solche Klima-Dynamiken simuliert und mit dem UTAC-Core interagiert, könnte wichtige Testfälle für resiliente Orientierungsarchitekturen bieten.
decision: |
    Wir werden ein neues Plugin `climate_thresholds` entwickeln, das als externes Modul mit dem UTAC-Core kommuniziert. Dieses Modul modelliert die gekoppelten Ozean- und Eisschildprozesse mithilfe eines einfachen Schwellenfeld-Ansatzes (Potentialmulden + Noise). Es wird in GenesisAeon als optionales Hilfsprogramm (05_Hilfsprogramme) geführt.
    - Abhängigkeiten: Basierend auf bestehenden Basis-Modulen (cosmic-moment, utac-core). Muss offline funktionsfähig sein (Regel 7) und per Blindtest validiert werden.
    - Metrics: Das Modul liefert RecoveryTime- und CSD-Indikatoren, um die Systemresilienz zu testen (siehe Forschungsfrage 001). Ergebnisse fließen in die UTAC-Core-Analyse ein.
consequences: |
    - Die Architektur muss einen klaren Datenpfad für Klimadaten vorsehen (ggf.
```

Rekordzeiten als Input).

- Eine Blindteststrategie für das Modul wird entwickelt (z.B. bekannte historische ENSO-Zyklen als Stimulus).
- Die Dokumentation (in `04\_Programme/` oder `05\_Hilfsprogramme/`) wird um dieses Modul erweitert.

Benchmark-Protokoll Trilayer/CREP (Skizze)

Für die Experimentreihe zur „**Forschungsfrage 001: Effekt strukturierter Kontextlayer**“ wird folgendes Protokoll empfohlen:

- **Versuchsaufbau:** Zwei (oder mehr) GPT-ähnliche Agenten interagieren über eine definierte Aufgabe (z.B. Problemlösung, Planung). Dabei werden die obigen Gruppen (Kontrolle, YAML, Trilayer, Trilayer+CREP) verglichen.
- **Störungen einführen:** Gelegentliche „Stör-Inputs“ (Themenwechsel, widersprüchliche Informationen) werden eingewebt.
- **Messphase:** Nach jeder Störung wird beobachtet, wie viele weitere Züge der Agenten benötigt werden, um wieder auf den ursprünglichen Gedankengang zu kommen – das ist die *Recovery Time* (siehe oben).
- **Kohärenz-Analyse:** Zusätzlich wird die Kohärenz (z.B. semantische Ähnlichkeit) der Antworten vor/ nach Störungen gemessen [39†L335-L341] .
- **Datenerfassung:** Alle Session-Protokolle werden geloggt. Wichtige Erkenntnisse und Zwischenstände als Trilayer-Report archiviert (z.B. in 02_Plaene/forschungsfrage-001-testprotokoll und 03_Architektur nach Abschluss).

Dies ermöglicht später statistische Auswertungen (Durchschnittswerte, Varianz, AC1-Autokorrelation) und Mensch/AI-Bewertungen der Verständlichkeit. Ein vergleichbarer Ansatz stammt aus der Klima-Frühwarnforschung, wo sich steigende Autokorrelation und längere Erholungszeiten als Signale für Resilienzverlust zeigen [46†L334-L342] – hier übertragen auf Konversation.

Quellen und Referenzen

- GenesisAeon Architektur-Planungsrepo (GitHub) [55†L319-L324] [55†L366-L375] [62†L283-L290] .
- STATUS.md (Ist-Stand und offene Tasks) [46†L300-L308] [46†L334-L342] .
- AR-Wirkung von strukturierter Kontexterweiterung: interne *DeepResearch*-Ergebnisse (Planungsdiskurse, Ideen) [39†L335-L341] [34†L222-L226] .
- Jenova AI Blog: „KI-Chat mit Verlauf“ – Kontextfenster-Limitierungen und persistentes Gedächtnis [84†L79-L86] . (Erklärt Bedeutung von externem Gedächtnis für LLMs, zitiert Serokell.)
- ADR-002 GenesisAeon Mission Statement [55†L335-L344] . (Leitidee: semantic maps, navigable paths, contextual structures.)
- Vereinfachende Analogie: Klimaschwellen (ENSO/Eisschild) als gekoppeltes Schwellenfeld [34†L222-L226] [46†L334-L342] . (Critical Slowing Down bei Glacier-Kipppunkten.)

<!-- Bitte beachten: Alle Quellennachweise wurden aus dem öffentlichen Architektur-Planungsrepo extrahiert und entsprechen dem Stand vom 26.6.2026. Nicht ausgewiesene Details beruhen auf Annahmen der Projektbeteiligten. -->